

Clean Code na Prática apostila

Nomes, funções e refatoração para quem escreve código todos os dias

por Carlos Ferreira

Material de estudo da plataforma MentorHub mentorhub.demosite.com

1. Nomes que explicam

O nome é a documentação mais lida do código. Variáveis e funções devem responder três perguntas: o que existe, por que existe e como é usado.

Antes e depois

Ruim: `const d = 30 // dias?`

Bom: `const diasParaEntrega = 30`

Ruim: `function calc(a, b)`

Bom: `function calcularDesconto(valorOriginal, percentualDesconto)`

Regras práticas

Evite abreviações criativas: `usr`, `mgr`, `tmp`. Se precisa de um comentário para explicar o nome, o nome está errado.

Use o vocabulário do domínio: em e-commerce, "carrinho" e "pedido" têm significado exato. Invente termos e o time inteiro paga a confusão.

Booleanos merecem prefixos: `isAtivo`, `temDesconto`, `podeEditar`. Uma leitura e o significado é claro.

2. Funções: uma coisa por vez

Uma função faz UMA coisa. Se você precisa de "e" para descrever o que ela faz ("valida e salva e envia e-mail"), ela precisa virar três.

O teste do nível de abstração

Todas as linhas de uma função devem viver no mesmo nível de detalhe. Misturar "calcularJuros()" com "req.headers.authorization.split()" na mesma função é o cheiro de refatoração.

Tamanho

Regra de bolso: se a função não cabe em uma tela sem rolar, considere quebrá-la. Funções curtas têm nomes claros, são testáveis isoladamente e reduzem bugs em metade.

Argumentos

Zero ou um argumento é o ideal. Três já é sinal de alerta; quatro ou mais, crie um objeto:

Ruim: criarPedido(nome, email, produto, qtd, preco, cupom)

Bom: criarPedido({ cliente, itens, cupom })

Efeitos colaterais

Uma função chamada "validarCPF" que também salva no banco quebra a confiança de quem lê. Efeitos precisam estar no nome ou fora da função.

3. Comentários honestos (e quando deletá-los)

O comentário ideal é o que você não precisou escrever porque o código se explicou. Na prática, comentários honestos caem em três casos:

1. Consequência intencional não óbvia: `// forçado a rodar antes do middleware de sessão`".
2. Esclarecimento de regra de negócio estranha: `// auditoria exige 2 anos, não 1`".
3. Aviso de dívida consciente: `// TODO: substituir quando a API v2 sair ticket #482`".

Comentários que devem ser DELETADOS

Ruído: `// incrementa i`" (o `i++` já disse isso).

Lixo comentado: blocos de código antigos comentados para isso existe o git. O histórico guarda tudo; o arquivo não precisa.

Comentário-mentira: o código mudou e o comentário ficou no tempo passado. Comentário errado é pior que nenhum: ele engana quem confia.

A regra de ouro

Antes de comentar, pergunte: dá para renomear, extrair função ou reestruturar para o código se explicar? Se sim, faça isso. O comentário é o último recurso, não o primeiro.

4. Refatoração sem medo

Refatorar é mudar a ESTRUTURA sem mudar o COMPORTAMENTO. A condição que torna isso seguro tem nome: testes. Sem testes, refatorar é operar no escuro.

O ciclo do refactoring

1. Escreva um teste que captura o comportamento atual.
2. Faça a mudança pequena (renomear, extrair função, mover método).
3. Rode os testes. Verde? Próxima mudança. Vermelho? Desfaça ou corrija.

Mudanças PEQUENAS e VERIFICADAS batem a grande reescrita "do zero" em 9 de cada 10 casos.

Os três refactorings mais rentáveis

Extrair função: bloco confuso vira função com nome que explica a intenção.

Renomear: nomes antigos que enganam viram nomes que contam a verdade.

Mover: código no lugar errado (regra de negócio no controller, SQL no componente) vai para a camada certa.

O hábito da regra do escoteiro

Deixe o código um pouco melhor do que encontrou: a cada visita a um arquivo, uma pequena melhoria. Seis meses disso transformam a base de código inteira sem sprint de refatoração, sem medo.